# Fantasy Draft Grader

**Midterm REST API | CY-421 Cloud Computing**

**Logan Pranger | October 30, 2026**

GitHub: https://github.com/LoganPrangerCT/Midterm

## 1. Why I built this

Every fantasy draft I have ever done ends in the same very annoying way. Me feeling like I crushed my draft, only to be told by ESPN that my team is deserving of a D-. That grade is given before a single point has been scored, it is unaffected by anything we later learn about the actual season, and it is never updated. Fast forward a few months, when the season has started and you know which players panned out, and now that my team is doing well I am missing the satisfaction of seeing a good grade.

That thought, along with the urge to stick it to ESPN, led me to create this project. The Fantasy Grader is a REST API that imports your Sleeper fantasy-football draft, scores every pick against 2026 rankings collected after the season had real information in it, and hands back a report card.

```
================================================================================
                          FANTASY DRAFT REPORT CARD
Team: loganpranger | Season: 2026
================================================================================

+------------------------------------------------------------------------------+
|                                                                              |
|                    +======================================+                  |
|                    |                                      |                  |
|                    |          DRAFT GRADE: [ C ]          |                  |
|                    |                                      |                  |
|                    +======================================+                  |
|                                                                              |
|               Total Value: +25.7 pts    Average Value: +1.977 pts            |
|                                                                              |
+------------------------------------------------------------------------------+

================================================================================
                             BIGGEST WINNERS
================================================================================
 Rank  Player Name                Round   Pick   Rank (Then vs Now)        Value
--------------------------------------------------------------------------------
  1.   Matthew Golden             Rnd 10  #98          #98 vs #30          +66.6
  2.   Bhayshul Tuten             Rnd 6   #58          #58 vs #26          +39.0
  3.   Jordyn Tyson               Rnd 13  #123        #123 vs #85          +10.6
--------------------------------------------------------------------------------

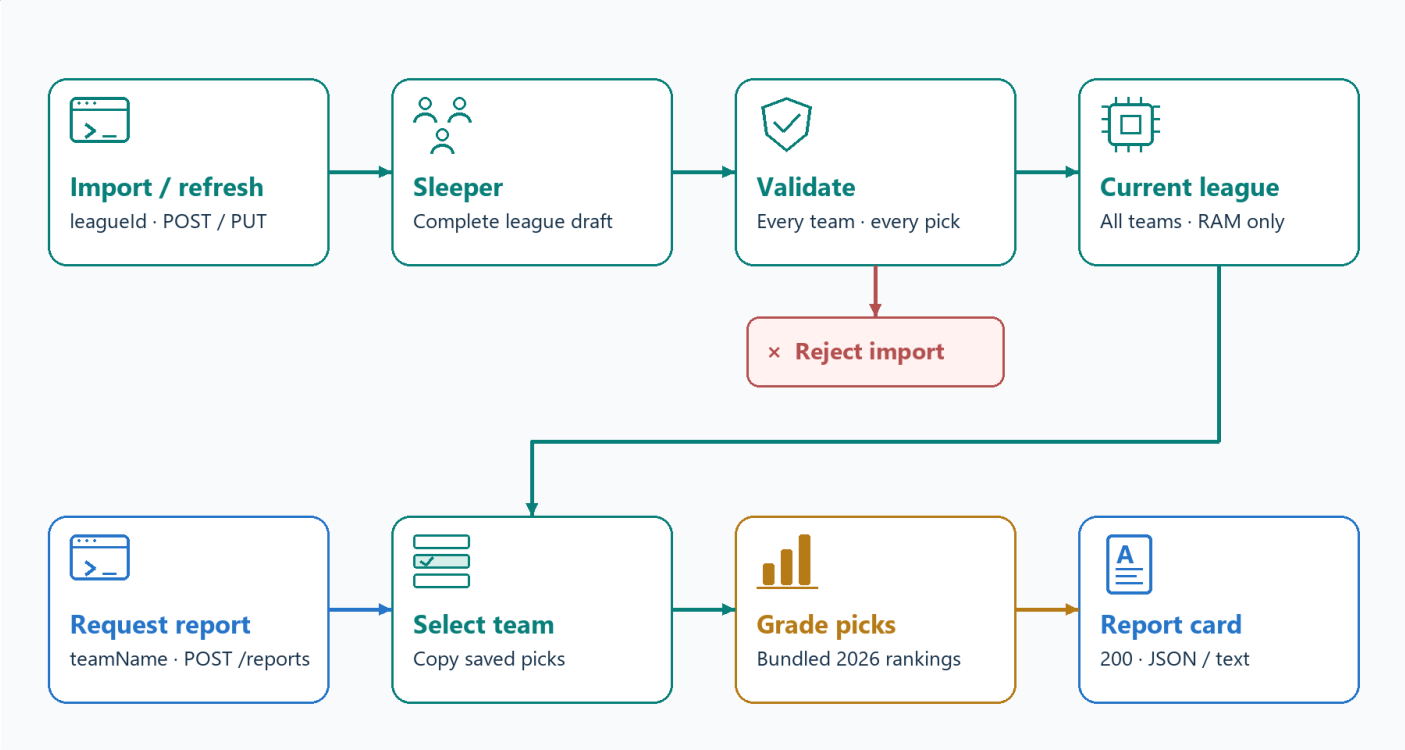
================================================================================
                             BIGGEST LOSERS
================================================================================
 Rank  Player Name                Round   Pick   Rank (Then vs Now)        Value
--------------------------------------------------------------------------------
  1.   Chase Brown                Rnd 2   #13          #13 vs #38          -42.2
  2.   Jonathon Brooks            Rnd 9   #83         #83 vs #186          -19.0
  3.   DeVonta Smith              Rnd 4   #38          #38 vs #57          -11.6
--------------------------------------------------------------------------------
```

## 2. How it is built

The implementation deliberately keeps the HTTP stuff separate from the football stuff. My reasoning was that cloud messaging, request validation and repository discipline should not be handled by the same files doing the Sleeper requests and team gradings. So the code separates into three layers: the Azure Functions definitions, a small repository that owns the one saved league, and a `fantasy/` package that does the importing, validating and grading.

| File or folder | Responsibility |
|---|---|
| `function_app.py` | Function app and its three HTTP triggers, key check, request parsing, responses |
| `repository.py` | The one current league: serial writes, defensive copies, UUIDs and timestamps |
| `fantasy/sleeper_client.py` | Read-only HTTPS calls to Sleeper with bounded timeouts |
| `fantasy/sleeper_import.py` | Validates the draft is complete, then normalizes all teams and picks |
| `fantasy/drafts.py`, `validation.py` | Team selection, report assembly and strict input checks |
| `fantasy/grading.py`, `report.py` | Pick scoring and report-card formatting |
| `fantasy/data/` | The 2026 rankings CSV files the grading engine reads |
| `scripts/`, `tests/` | Local startup, offline tests and hosted smoke test |

The diagram below shows the import and report flows.



## 3. Designing the resource

The resource is one **league snapshot**, and I want to explain the two decisions inside that name. First, "league" rather than something smaller like a report: the report card is a pure calculation, so storing reports as resources would create an update problem that does not actually exist. The report is computed whenever it is asked for. Second, "snapshot": the whole league is imported and validated in one shot, or nothing is saved.

Only one current league is stored at a time. That was a scope decision for the midterm to make the API simpler to explain as well: there is exactly one thing to read, refresh or delete, so every ambiguous case collapses. A second create returns 409 until the caller updates or deletes the league it already has.

| Field | Purpose |
|---|---|
| `id` | Generated UUID used in PUT and DELETE paths |
| `createdAt` / `updatedAt` | UTC timestamps; a refresh preserves `createdAt` |
| `leagueId` / `leagueName` | Sleeper identity and the readable league name |
| `draftId` / `season` | Selected completed draft and the supported season (2026) |
| `rosters` / `users` | Manager identities and roster lookup information |
| `teams` | Every team with rosterId, teamName, season and picks |
| `picks` | Overall pick number, round, player and position per selection |

### *The CRUD requirements*

The assignment outlines what the four operations have to do, and here is where each one lives.

| Required operation | How Fantasy Lab meets it |
|---|---|
| Create (POST) | `POST /api/leagues` imports the full draft, validates it end to end, and only then saves the snapshot |
| Read (GET) | `GET /api/leagues` returns the stored snapshot in an array, or when nothing is saved, straight from memory |
| Update (PUT) | `PUT /api/leagues/{id}` re-imports and replaces the saved snapshot under the same UUID |
| Delete (DELETE) | `DELETE /api/leagues/{id}` removes the snapshot |
| Unique identifier | A generated UUID returned by create, used in update/delete paths, independent of the upstream Sleeper ID |

# 4. Using the API

Base URL (Azure): https://fantasy-midterm-lpranger-degda0fabcbehhch.northcentralus-01.azurewebsites.net/api

**Local:**
`http://127.0.0.1:7071/api`

Every request requires the API key in a header. The placeholder below is a placeholder, not the real key. Send `Content-Type: application/json` with JSON bodies; responses are JSON by default, and no query parameters are used anywhere.

```
x-api-key: <APP_API_KEY>
Content-Type: application/json
```

There are five routes. The first four are the CRUD lifecycle of the saved league; the fifth computes reports.

| Method | Route | Success response |
|---|---|---|
| POST | `/api/leagues` | 201, saved snapshot and Location header |
| GET | `/api/leagues` | 200, current snapshot in an array, or `[]` |
| PUT | `/api/leagues/{id}` | 200, refreshed or replaced snapshot |
| DELETE | `/api/leagues/{id}` | 200, confirmation that the snapshot is gone |
| POST | `/api/reports` | 200, one manager's report card (JSON or plain text) |

### *Posting in: save the league*

Send only the Sleeper league ID. The service figures out the draft, rosters and picks by making a request to the Sleeper API, and retrieves the full draft board for the most recent year.

```
POST /api/leagues
{"leagueId":"1391859959213359104"}
```

What you get back is 201 Created, a Location header pointing at the new snapshot, and the resource itself. This excerpt from my hosted test shows two actual picks. Ellipses mark omitted rosters, users, picks and teams:

```
{"leagueId":"1391859959213359104","leagueName":"Family league",
 "draftId":"1391859960396140544","season":2026,
 "id":"4170f7da-f61d-4fb8-b75b-14695383fb13",
 "createdAt":"2026-11-04T00:43:03.287054Z",
 "updatedAt":"2026-11-04T00:43:03.287054Z",
 ...
 "teams":[{"rosterId":2,"teamName":"loganpranger","season":2026,
   "picks":[
     {"overallPick":8,"round":1,"playerName":"James Cook","position":"RB"},
     {"overallPick":13,"round":2,"playerName":"Chase Brown","position":"RB"},
     ...
   ]},
   ...
 ]}
```

Trying to save a second league returns 409; update or delete the existing one first. Other errors: 400 for malformed input or an unsupported season, 401 for a bad key, 404 when Sleeper has no such data, or 502/503/504 if the upstream connection itself fails.

### Reading it back

GET /api/leagues takes no body and reads only memory, so it never waits on Sleeper. With nothing saved you get []; after the create above you get that snapshot in an array:

```
GET /api/leagues

[{"id":"4170f7da-f61d-4fb8-b75b-14695383fb13",
  "leagueId":"1391859959213359104","leagueName":"Family league"}]
```

### *Updating: refresh or replace*

PUT /api/leagues/{id} refreshes the saved Sleeper data or replaces it with a different league. The path ID is the saved snapshot UUID; the body contains the Sleeper leagueId. Reusing that leagueId refreshes the snapshot; supplying a different one replaces its contents under the same UUID. Rankings remain the bundled CSV and change only when that file is updated.

PUT /api/leagues/4170f7da-f61d-4fb8-b75b-14695383fb13

`{"leagueId":"1391859959213359104"}`

{"id":"4170f7da-f61d-4fb8-b75b-14695383fb13",

"leagueId":"1391859959213359104","leagueName":"Family league",
"createdAt":"2026-11-04T00:43:03.287054Z",
"updatedAt":"2026-11-04T00:49:59.322522Z"}

The response returned 200 with the UUID and `createdAt` unchanged and a new `updatedAt`, which is exactly the expected output. If the fresh import fails, the old snapshot survives too. Errors: 400 for an invalid UUID or body, 401, 404 for an unknown UUID or missing upstream data, and the upstream failures above.

### *Deleting it*

`DELETE /api/leagues/{id}` removes the snapshot and everything under it:

DELETE /api/leagues/4170f7da-f61d-4fb8-b75b-14695383fb13

{"id":"4170f7da-f61d-4fb8-b75b-14695383fb13","deleted":true}

After this, team-name-only reports return 404 until someone saves a league again, and deleting the same UUID twice correctly returns 404 the second time.

### *Getting a report card*

`POST /api/reports` is why anyone would use this API, and it has two modes. The saved mode sends just a `teamName`, which can be a team name, a manager's display name, a username or a user ID, matched case-insensitively, and the report is computed purely from stored picks with zero Sleeper calls. Success is 200 rather than 201, deliberately, because no report is stored.

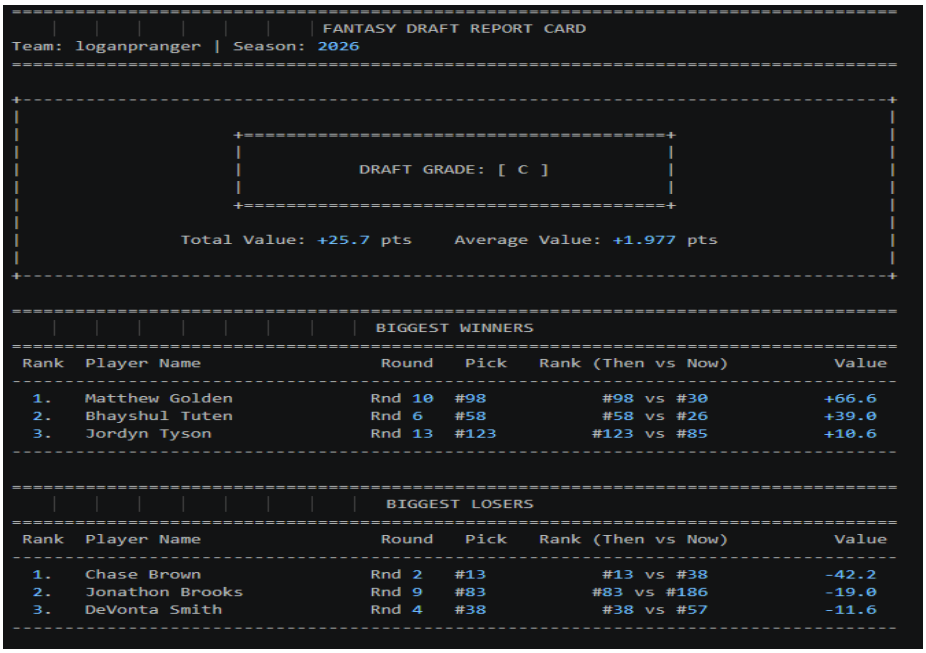
```
POST /api/reports
{"teamName":"loganpranger"}
```

```
{"rosterId":2,"teamName":"loganpranger","season":2026,
 "draftGrade":"C",
 "picks":["...every pick with its scored value..."],
 "evaluation":"...coverage, totals, average, best and worst picks...",
 "rankingsVersion":"2026-sha256-..."}
```

There's also a simpler way to get a report card: add leagueId to the body. Then the service fetches fresh Sleeper data, grades the named manager, and leaves the saved league untouched:

```
POST /api/reports
Accept: text/plain
{"leagueId":"1391859959213359104","teamName":"loganpranger"}
```

The text response is the actual report card, and here is a condensed excerpt of the real one captured from the hosted API on October 30, for my own draft:



```
                                FANTASY DRAFT REPORT CARD
Team: loganpranger | Season: 2026

                                DRAFT GRADE: [ C ]

                Total Value: +25.7 pts    Average Value: +1.977 pts

                                  BIGGEST WINNERS
 Rank  Player Name                Round   Pick   Rank (Then vs Now)          Value
  1.   Matthew Golden             Rnd 10  #98          #98 vs #30           +66.6
  2.   Bhayshul Tuten             Rnd 6   #58          #58 vs #26           +39.0
  3.   Jordyn Tyson               Rnd 13  #123        #123 vs #85           +10.6

                                  BIGGEST LOSERS
 Rank  Player Name                Round   Pick   Rank (Then vs Now)          Value
  1.   Chase Brown                Rnd 2   #13          #13 vs #38           -42.2
  2.   Jonathon Brooks            Rnd 9   #83          #83 vs #186          -19.0
  3.   DeVonta Smith              Rnd 4   #38          #38 vs #57           -11.6
```

After all this, my actual grade is better than what ESPN gave me at the start of the season.

### *Possible Error Codes*

Invalid bodies return 400 for unknown fields, duplicate keys, non-finite numbers, oversized bodies or invalid team selectors. Errors use the same JSON structure across every route:

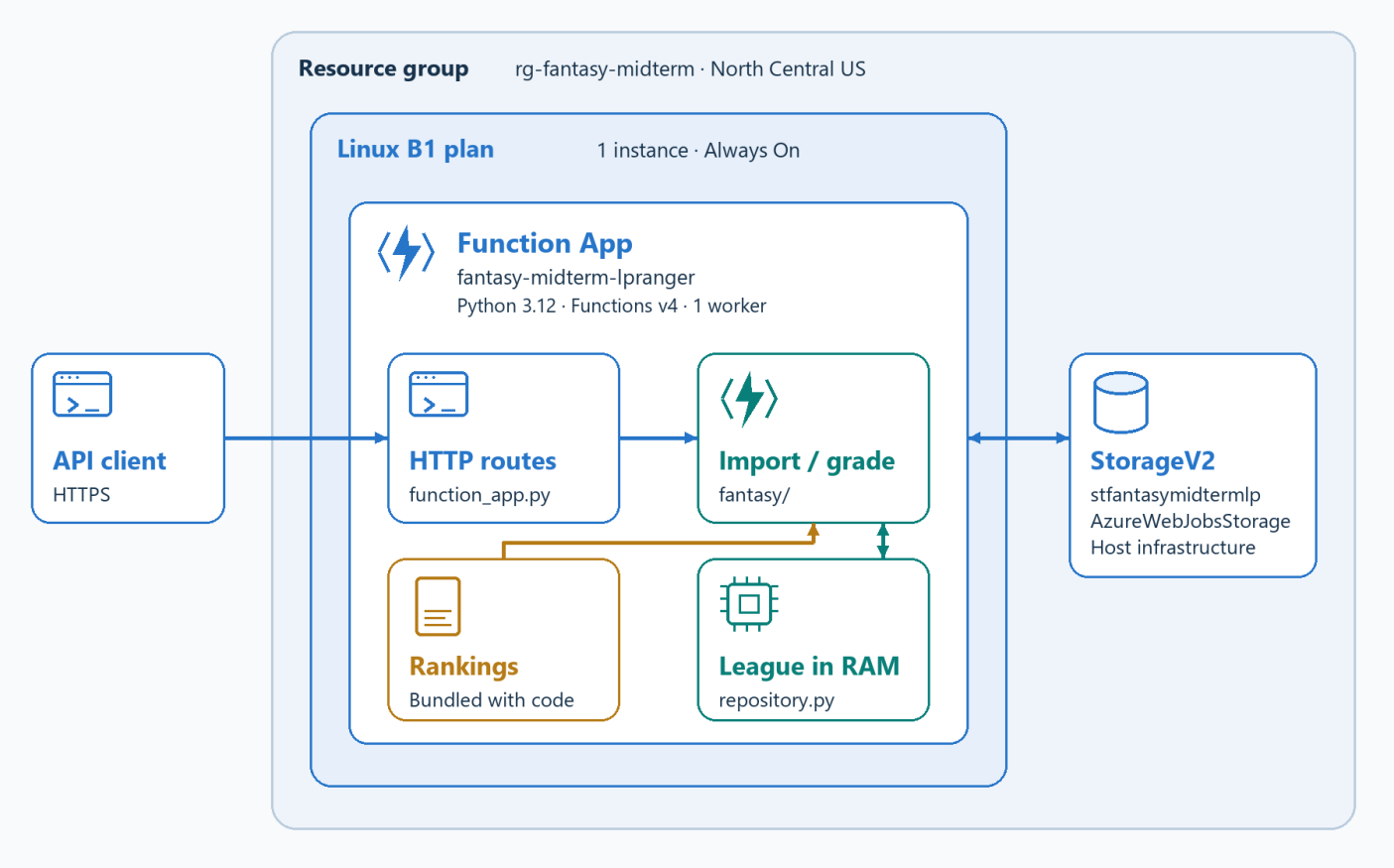
```
{"error":{"code":"validation_error","message":"body missing fields: leagueId"}}
```

| Status | Error code | Meaning |
|---|---|---|
| 400 | `validation_error` | Invalid body, UUID, team selector or season |
| 401 | `unauthorized` | Missing or incorrect `x-api-key` |
| 404 | `not_found` | Missing saved resource or team, or missing upstream data |
| 409 | `conflict` | A league is already saved |
| 502 | `upstream_error` | Sleeper connection failed or returned unusable data |
| 503 | `upstream_error` / `configuration_error` | Sleeper rate limit or unconfigured app key |
| 504 | `upstream_error` | Sleeper timed out |
| 500 | `internal_error` | Unexpected failure; details are logged server-side only |

Authentication runs first. An existing saved league makes a second create return 409 before its body is parsed. PUT checks the saved UUID before reading the body. A team-name-only report returns 404 when no league is saved, before checking its fields. Duplicate ranking names are excluded from grading with an ambiguous warning rather than selecting a player arbitrarily.

# 5. Deploying to Azure

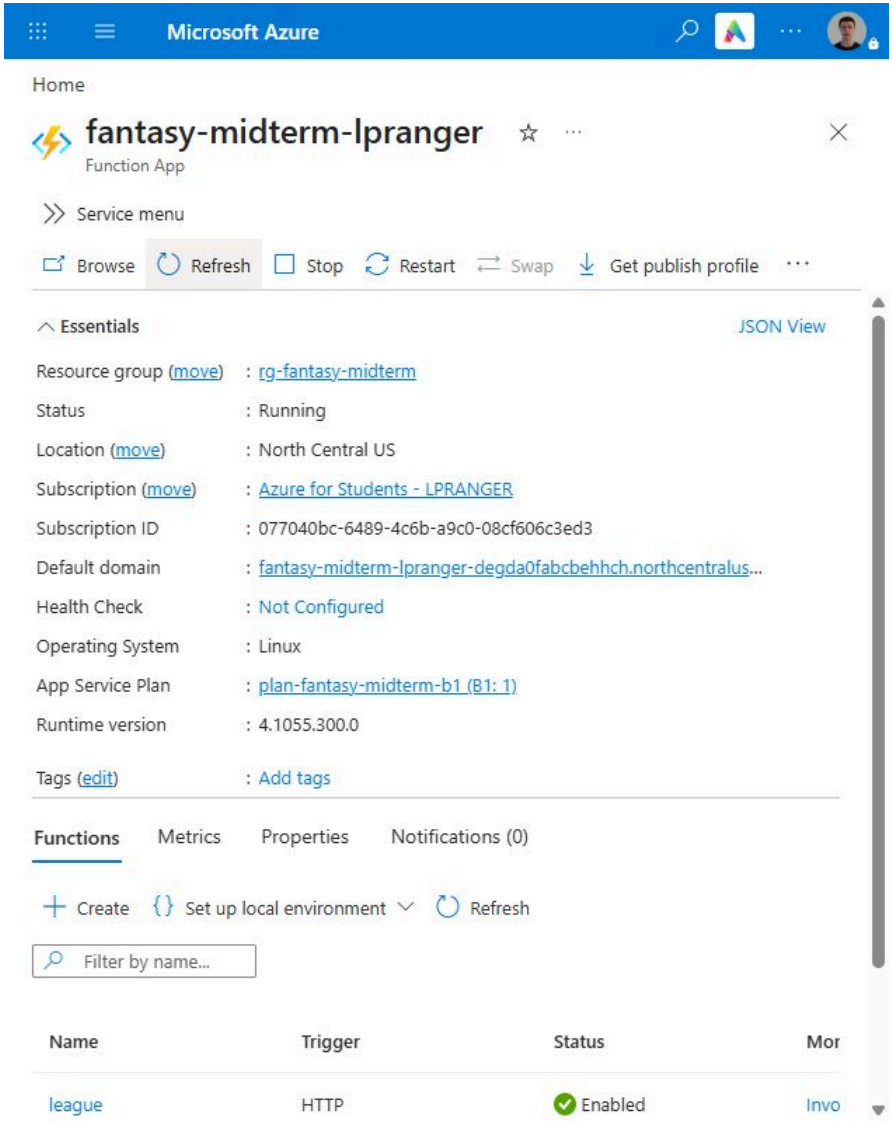
The API is built on 3 parts: a Basic B1 App Service plan for the compute, the Function App that runs the Python code, and a storage account the Functions host uses for its own infrastructure. This implementation is not the cheapest possible since I decided not to use the flex option to avoid the wait time of a cold start. Despite this, the 24/7 uptime only has a cost of roughly $0.43 per day

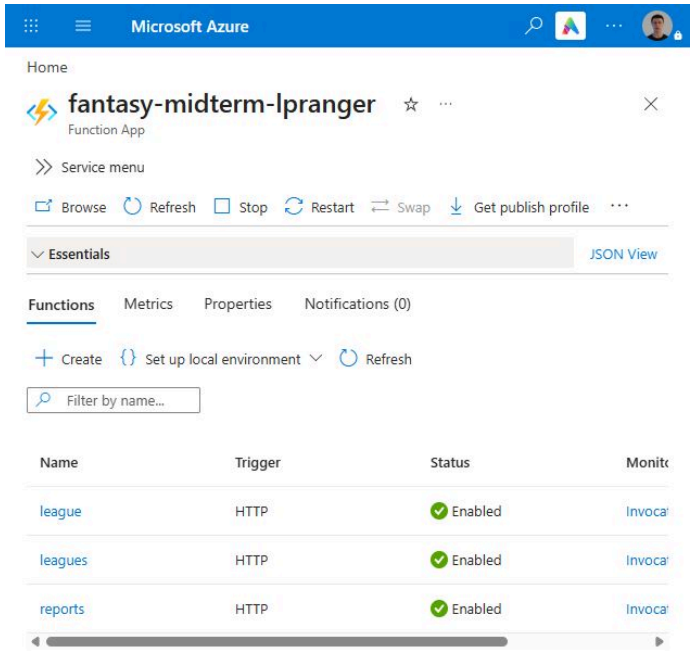



The plan supplies the compute; the Function App runs the HTTP triggers, import and grading; storage exists for the host. League state lives only in Python memory.

| Resource | Deployed name and role |
|---|---|
| Resource group | `rg-fantasy-midterm`; one lifecycle and management scope |
| Linux Basic B1 plan | `plan-fantasy-midterm-b1`; one compute instance |
| Function App | `fantasy-midterm-lpranger`; Python 3.12, Functions ~4 |
| Storage Standard_LRS | `stfantasymidtermlp`; AzureWebJobsStorage for the host [5] |

The Azure portal confirms what the deployment should look like. Below on the left, the Function App overview: running on Linux, `plan-fantasy-midterm-b1`, North Central US, with the familiar default hostname. On the right, the function list: exactly `league`, `leagues` and `reports`, all enabled, with no stale routes left behind.



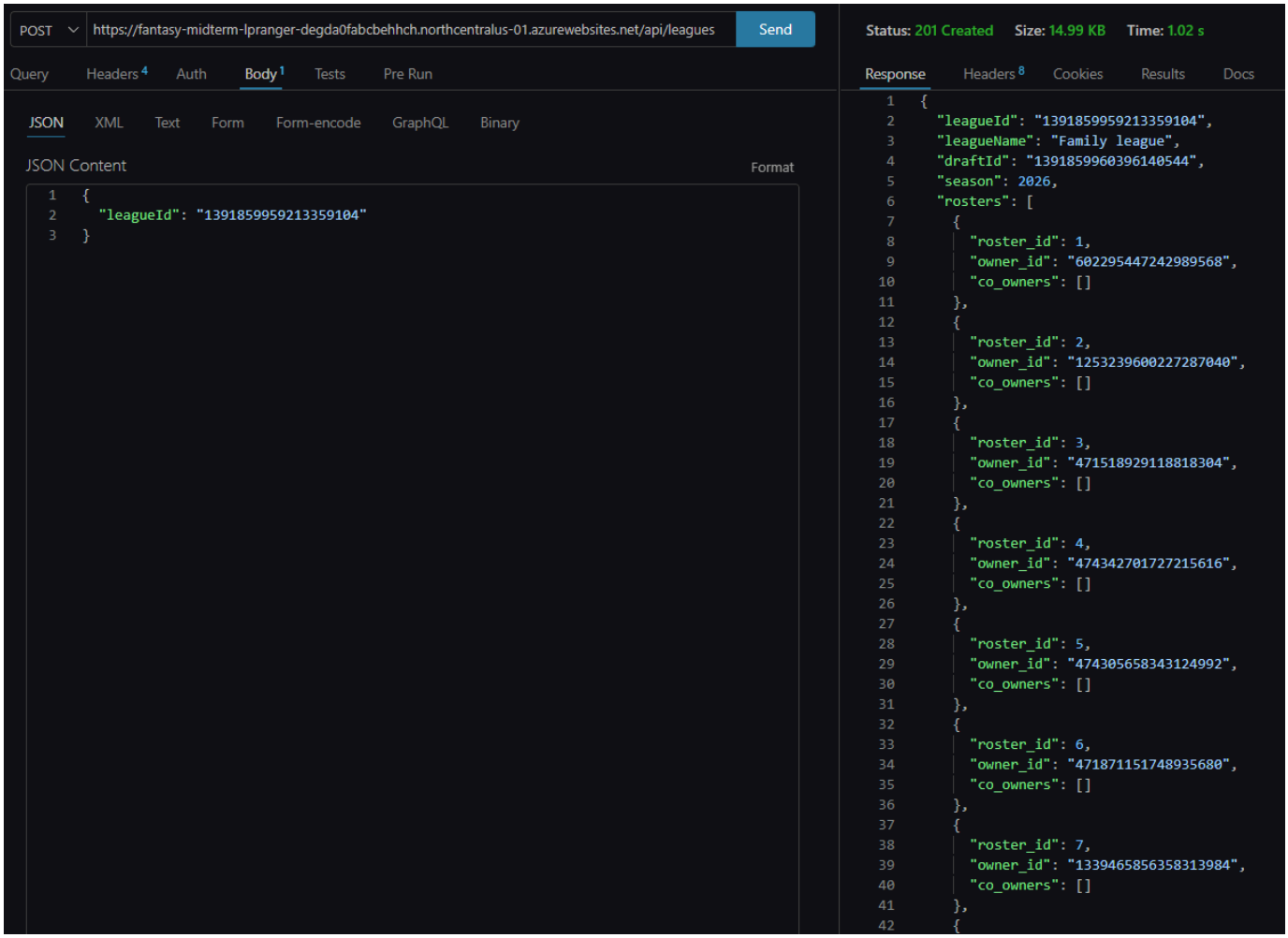
# 6. Testing the hosted API

### *T1. Create: saving the league*

Endpoint / method: POST https://fantasy-midterm-lpranger-degda0fabcbehhch.northcentralus-01.azurewebsites.net/api/leagues

**Expected:** 201 Created with the saved snapshot reflective of the Sleeper league, its generated UUID and the Location header.

**Actual:** 201 Created. The response shows `leagueName: "Family league"`, the draft ID, season, and the roster array.



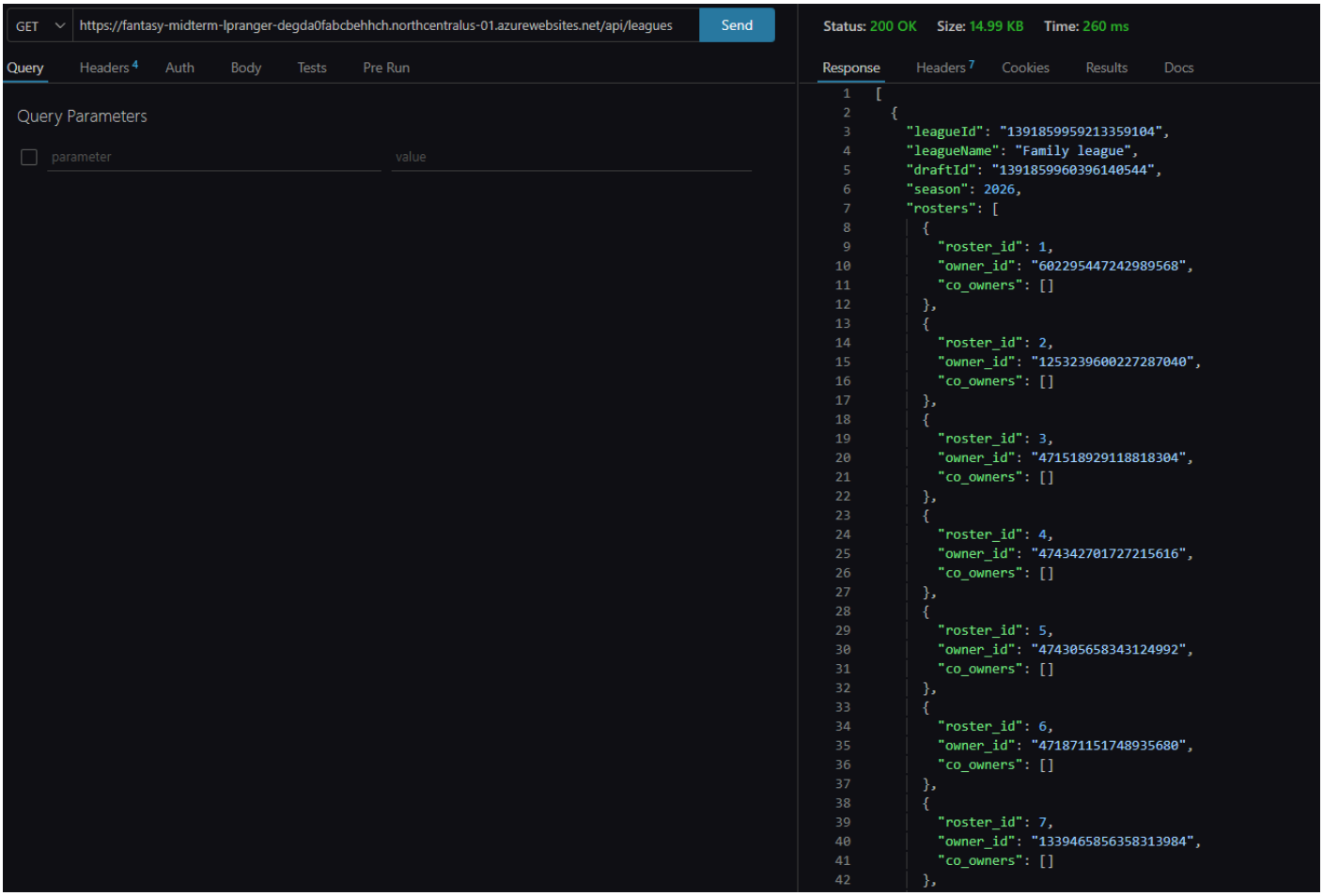

Result: PASS - the saved UUID returned here drives the update and delete tests below.

### *T2. Read: getting it back*

Endpoint / method: GET https://fantasy-midterm-lpranger-degda0fabcbehhch.northcentralus-01.azurewebsites.net/api/leagues

**Expected:** 200 with the entire snapshot in an array.

**Actual:** 200 OK; the response is an array whose first element is the saved league.



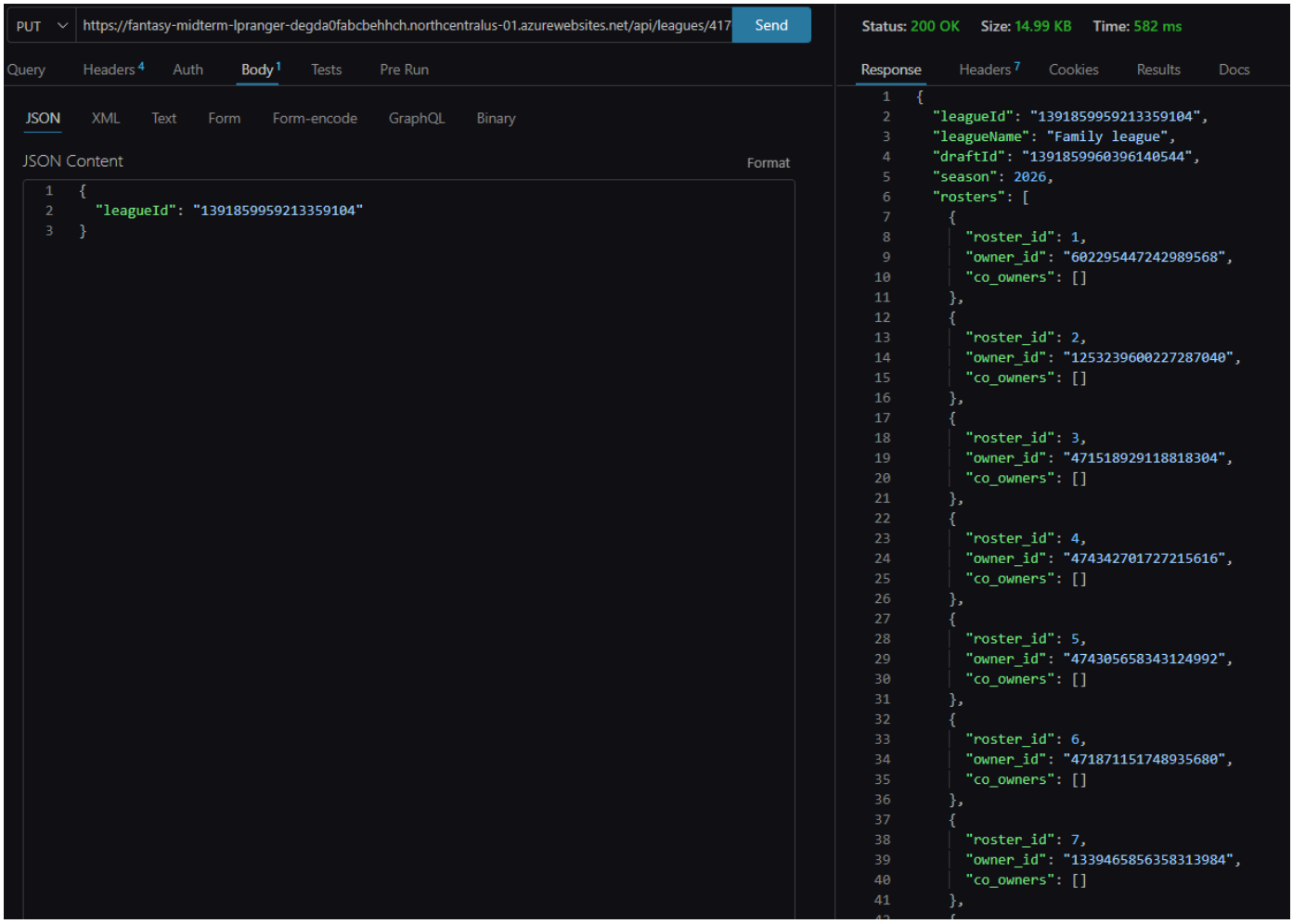

Result: PASS - the array wraps the saved snapshot; the read makes no Sleeper calls.

### *T3. Update: refreshing the same league*

Endpoint / method: PUT
https://fantasy-midterm-lpranger-degda0fabcbehhch.northcentralus-01.azurewebsites.net/api/leagues/4170f7da-f61d-4fb8-b75b-14695383fb13

**Expected:** 200 with the same UUID and `createdAt`, a new `updatedAt`.

**Actual:** 200 OK; the request path targets the saved UUID and the response is the refreshed league.



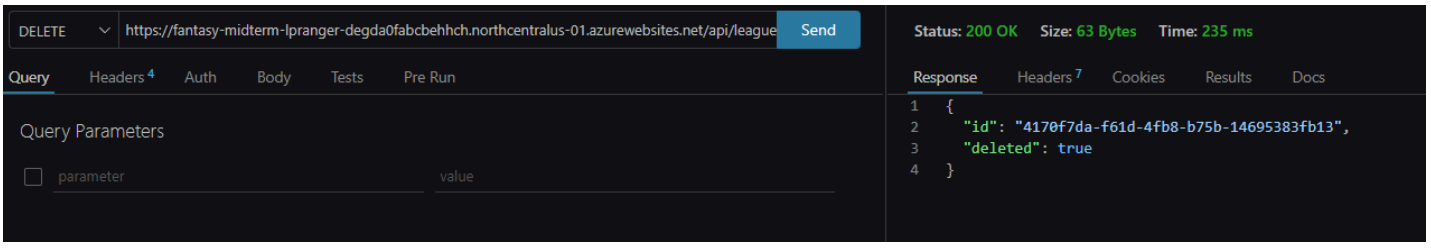

Result: PASS - same UUID and createdAt with a fresh updatedAt: a refresh, not a create.

### *T4. Delete: removing it*

Endpoint / method: DELETE
https://fantasy-midterm-lpranger-degda0fabcbehhch.northcentralus-01.azurewebsites.net/api/leagues/4170f7da-f61d-4fb8-b75b-14695383fb13

**Expected:** 200 with the same UUID and `deleted: true`.

**Actual:** 200 OK
Result: PASS - the replacement id matches the saved UUID; T7 repeats the delete and meets 404.



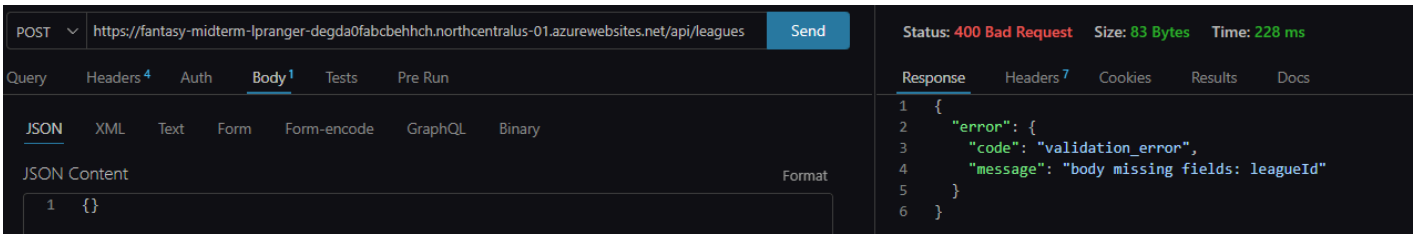
### *T5. Bad request: a create with nothing in it*

Endpoint / method: POST https://fantasy-midterm-lpranger-degda0fabcbehhch.northcentralus-01.azurewebsites.net/api/leagues

**Expected:** 400 `validation_error`, since the body is `{}` with no leagueId.

**Actual:** 400 Bad Request; the response is the structured error `body missing fields: leagueId`.

Result: PASS - validation rejects the empty body before any Sleeper call is attempted.

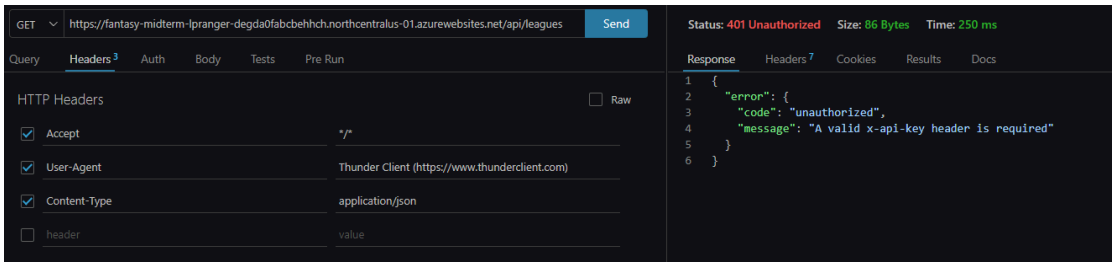



### *T6. Unauthorized: no API key*

Endpoint / method: GET https://fantasy-midterm-lpranger-degda0fabcbehhch.northcentralus-01.azurewebsites.net/api/leagues

**Expected:** 401 `unauthorized`.

**Actual:** 401 Unauthorized; the response is the structured error with the message "A valid x-api-key header is required".



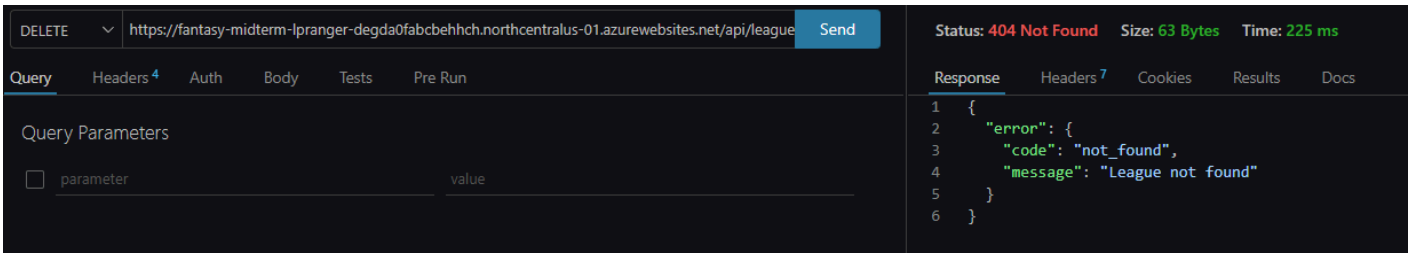

### *T7. Not found: deleting what is already gone*

Endpoint / method: DELETE
https://fantasy-midterm-lpranger-degda0fabcbehhch.northcentralus-01.azurewebsites.net/api/leagues/4170f7da-f61d-4fb8-b75b-14695383fb13

**Expected:** 404 `not_found`, because T4 already removed the snapshot.

**Actual:** 404 Not Found; the response is the structured error "League not found"

Result: PASS - the 404 after deletion confirms the removal took effect immediately.




Result: PASS - the key travels only in the x-api-key header and is never displayed.

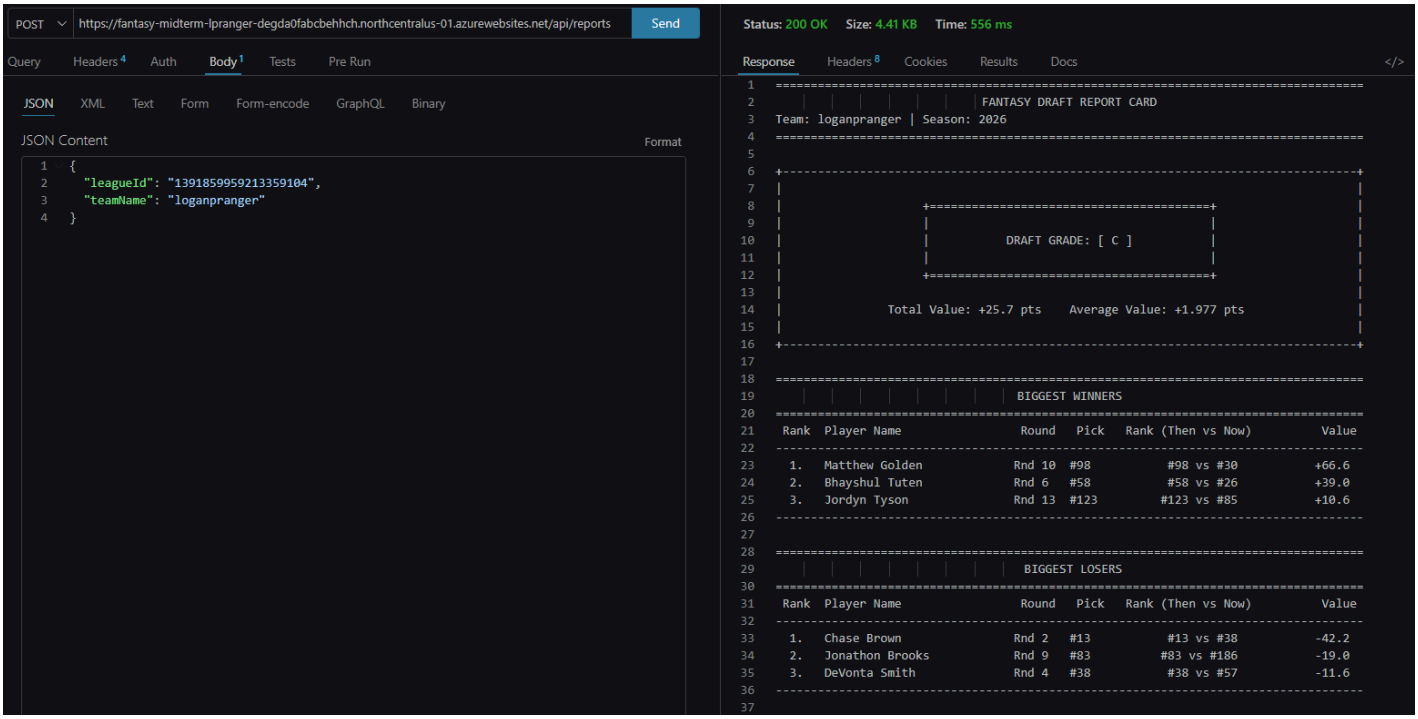
### *T8. The one-request demo: full report card, plain text*

Endpoint / method: POST https://fantasy-midterm-lpranger-degda0fabcbehhch.northcentralus-01.azurewebsites.net/api/reports

**Request:** `{"leagueId":"1391859959213359104","teamName":"loganpranger"}`, valid key, `Accept: text/plain`.

**Expected:** 200 with a readable report card for the named manager, computed from a fresh import, without creating or altering saved state.

**Actual:** 200. The response identifies loganpranger, season 2026, grade C, total value +25.7 , average +1.977 , with winners, losers and the full draft board (the condensed excerpt in section 4 came from this response).




Result: PASS - this request used a fresh import and left the saved league untouched.

## References


[1] Microsoft Learn. Azure Functions Python reference.

[2] Microsoft Learn. Azure RBAC overview.

[3] Microsoft Learn. Disable basic publishing authentication.

[4] Microsoft Learn. Functions dedicated hosting.

[5] Microsoft Learn. Functions storage considerations.